EXPERIMENT-02

Candidate-Elimination Algorithm: Dataset-Based Python Programs

Name: MURE THEJASRI Reg.No: 192311298

2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Problem 1: Enjoying Sport

Aim

To implement the Candidate-Elimination algorithm on a sport-related dataset stored in a CSV file and output the set of all hypotheses consistent with the training examples (the version space).

Procedure

1. Save the training examples in sport_data.csv with the target class (Yes / No) in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV rows; initialize S with the first positive example and G with the all-'?' hypothesis.
4. For a positive example, replace the attributes of S that differ with '?' and remove from G the hypotheses that do not cover the example.
5. For a negative example, specialize every hypothesis in G that covers it using the values in S, and drop any hypothesis that is more general than another member of G.
6. Print S and G after each example; the last line gives the final version space.

Dataset File / Input

File name: sport_data.csv

The CSV file contains a header row followed by these training examples:

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Code

```
import csv

data = list(csv.reader(open("sport_data.csv")))[1:]
n = len(data[0]) - 1
S = data[0][:-1]                    # first example is positive
G = [["?"] * n]

for row in data:
    x = row[:-1]
    if row[-1] == "Yes":
        S = [S[i] if S[i] == x[i] else "?" for i in range(n)]
        G = [g for g in G if all(g[i] in ("?", x[i]) for i in range(n))]
    else:
        new = []
        for g in G:
            if all(g[i] in ("?", x[i]) for i in range(n)):
                for i in range(n):
                    if g[i] == "?" and S[i] not in ("?", x[i]):
                        h = g.copy(); h[i] = S[i]; new.append(h)
```

```
            else:
                new.append(g)
        G = [g for g in new if not any(t != g and all(t[i] in ('?', g[i]) for i in range(n)) for t in new)]
    print("S =", S, " G =", G)
```

Output

```
S = ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']  G = [['?', '?', '?', '?', '?', '?']]
S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']  G = [['?', '?', '?', '?', '?', '?']]
S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']  G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['
S = ['Sunny', 'Warm', '?', 'Strong', '?', '?']  G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]
```

Result

The version space is bounded by S = ['Sunny', 'Warm', '?', 'Strong', '?', '?'] and G = {['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']}. The six hypotheses lying between these boundaries are all consistent with the four training examples.

Problem 2: Choosing a Laptop

Aim

To apply the Candidate-Elimination algorithm to a laptop-selection dataset stored in a CSV file and find the specific and general boundaries of the version space.

Procedure

1. Save the training examples in laptop_data.csv with the target class (Yes / No) in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV rows; initialize S with the first positive example and G with the all-'?' hypothesis.
4. For a positive example, replace the attributes of S that differ with '?' and remove from G the hypotheses that do not cover the example.
5. For a negative example, specialize every hypothesis in G that covers it using the values in S, and drop any hypothesis that is more general than another member of G.
6. Print S and G after each example; the last line gives the final version space.

Dataset File / Input

File name: laptop_data.csv

The CSV file contains a header row followed by these training examples:

Budget	Weight	Battery	Purpose	Selected
High	Light	Long	Study	Yes
High	Light	Short	Work	Yes
Low	Heavy	Short	Gaming	No
Medium	Light	Long	Study	No
High	Light	Long	Work	Yes

Code

```
import csv

data = list(csv.reader(open("laptop_data.csv")))[1:]
n = len(data[0]) - 1
S = data[0][:-1]                    # first example is positive
G = [["?"] * n]

for row in data:
    x = row[:-1]
    if row[-1] == "Yes":
        S = [S[i] if S[i] == x[i] else "?" for i in range(n)]
        G = [g for g in G if all(g[i] in ("?", x[i]) for i in range(n))]
    else:
        new = []
        for g in G:
```

```
            if all(g[i] in ("?", x[i]) for i in range(n)):
                for i in range(n):
                    if g[i] == "?" and S[i] not in ("?", x[i]):
                        h = g.copy(); h[i] = S[i]; new.append(h)
            else:
                new.append(g)
        G = [g for g in new if not any(t != g and all(t[i] in ('?', g[i]) for i in range(n)) for t in new)]
    print("S =", S, " G =", G)
```

Output

```
S = ['High', 'Light', 'Long', 'Study']  G = [['?', '?', '?', '?']]
S = ['High', 'Light', '?', '?']  G = [['?', '?', '?', '?']]
S = ['High', 'Light', '?', '?']  G = [['High', '?', '?', '?'], ['?', 'Light', '?', '?']]
S = ['High', 'Light', '?', '?']  G = [['High', '?', '?', '?']]
S = ['High', 'Light', '?', '?']  G = [['High', '?', '?', '?']]
```

Result

The negative example (Medium, Light, Long, Study) removes ['?', 'Light', '?', '?'] from G, leaving S = ['High', 'Light', '?', '?'] and G = {['High', '?', '?', '?']}. Only two hypotheses remain consistent with the data.

Problem 3: Plant Growth

Aim

To use the Candidate-Elimination algorithm on a plant-growth dataset stored in a CSV file and determine all hypotheses consistent with the growing conditions observed.

Procedure

1. Save the training examples in plant_data.csv with the target class (Yes / No) in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV rows; initialize S with the first positive example and G with the all-'?' hypothesis.
4. For a positive example, replace the attributes of S that differ with '?' and remove from G the hypotheses that do not cover the example.
5. For a negative example, specialize every hypothesis in G that covers it using the values in S, and drop any hypothesis that is more general than another member of G.
6. Print S and G after each example; the last line gives the final version space.

Dataset File / Input

File name: plant_data.csv

The CSV file contains a header row followed by these training examples:

Soil	Sunlight	Watering	Season	PlantGrows
Loamy	Full	Moderate	Spring	Yes
Loamy	Partial	Moderate	Spring	Yes
Sandy	Full	Low	Summer	No
Clay	Partial	Frequent	Autumn	No
Loamy	Full	Moderate	Summer	Yes

Code

```
import csv

data = list(csv.reader(open("plant_data.csv")))[1:]
n = len(data[0]) - 1
S = data[0][:-1]                    # first example is positive
G = [["?"] * n]

for row in data:
    x = row[:-1]
    if row[-1] == "Yes":
        S = [S[i] if S[i] == x[i] else "?" for i in range(n)]
        G = [g for g in G if all(g[i] in ("?", x[i]) for i in range(n))]
```

```
    else:
        new = []
        for g in G:
            if all(g[i] in ("?", x[i]) for i in range(n)):
                for i in range(n):
                    if g[i] == "?" and S[i] not in ("?", x[i]):
                        h = g.copy(); h[i] = S[i]; new.append(h)
            else:
                new.append(g)
        G = [g for g in new if not any(t != g and all(t[i] in ('?', g[i]) for i in range(n)) for t in new)]
    print("S =", S, " G =", G)
```

Output

```
S = ['Loamy', 'Full', 'Moderate', 'Spring']  G = [['?', '?', '?', '?']]
S = ['Loamy', '?', 'Moderate', 'Spring']  G = [['?', '?', '?', '?']]
S = ['Loamy', '?', 'Moderate', 'Spring']  G = [['Loamy', '?', '?', '?'], ['?', '?', 'Moderate', '?'], ['?', '?', '?', 'Spring']]
S = ['Loamy', '?', 'Moderate', 'Spring']  G = [['Loamy', '?', '?', '?'], ['?', '?', 'Moderate', '?'], ['?', '?', '?', 'Spring']]
S = ['Loamy', '?', 'Moderate', '?']  G = [['Loamy', '?', '?', '?'], ['?', '?', 'Moderate', '?']]
```

Result

The final boundaries are S = ['Loamy', '?', 'Moderate', '?'] and G = {['Loamy', '?', '?', '?'], ['?', '?', 'Moderate', '?']}. Three hypotheses are consistent: soil = Loamy, watering = Moderate, or both together.

Problem 4: Going on a Trip

Aim

To implement Candidate-Elimination on a travel-decision dataset stored in a CSV file and observe whether the version space converges to a single hypothesis.

Procedure

1. Save the training examples in travel_data.csv with the target class (Yes / No) in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV rows; initialize S with the first positive example and G with the all-'?' hypothesis.
4. For a positive example, replace the attributes of S that differ with '?' and remove from G the hypotheses that do not cover the example.
5. For a negative example, specialize every hypothesis in G that covers it using the values in S, and drop any hypothesis that is more general than another member of G.
6. Print S and G after each example; the last line gives the final version space.

Dataset File / Input

File name: travel_data.csv

The CSV file contains a header row followed by these training examples:

Weather	Distance	Transport	Companions	GoTrip
Clear	Short	Train	Friends	Yes
Clear	Long	Car	Friends	Yes
Stormy	Short	Bus	Alone	No
Clear	Short	Train	Family	No
Cloudy	Long	Car	Friends	Yes

Code

```
import csv

data = list(csv.reader(open("travel_data.csv")))[1:]
n = len(data[0]) - 1
S = data[0][:-1]                      # first example is positive
G = [["?"] * n]

for row in data:
    x = row[:-1]
    if row[-1] == "Yes":
        S = [S[i] if S[i] == x[i] else "?" for i in range(n)]
```

```
        G = [g for g in G if all(g[i] in ("?", x[i]) for i in range(n))]
    else:
        new = []
        for g in G:
            if all(g[i] in ("?", x[i]) for i in range(n)):
                for i in range(n):
                    if g[i] == "?" and S[i] not in ("?", x[i]):
                        h = g.copy(); h[i] = S[i]; new.append(h)
            else:
                new.append(g)
        G = [g for g in new if not any(t != g and all(t[i] in ('?', g[i]) for i in range(n)) for t in new)]
    print("S =", S, " G =", G)
```

Output

```
S = ['Clear', 'Short', 'Train', 'Friends']  G = [['?', '?', '?', '?']]
S = ['Clear', '?', '?', 'Friends']  G = [['?', '?', '?', '?']]
S = ['Clear', '?', '?', 'Friends']  G = [['Clear', '?', '?', '?'], ['?', '?', '?', 'Friends']]
S = ['Clear', '?', '?', 'Friends']  G = [['?', '?', '?', 'Friends']]
S = ['?', '?', '?', 'Friends']  G = [['?', '?', '?', 'Friends']]
```

Result

The S and G boundaries converge to the same hypothesis, ['?', '?', '?', 'Friends']. The target concept is learned exactly: the trip is chosen whenever the person travels with friends.

Problem 5: Choosing a Fitness Activity

Aim

To apply the Candidate-Elimination algorithm to a fitness-activity dataset stored in a CSV file and output the complete set of consistent hypotheses.

Procedure

1. Save the training examples in fitness_data.csv with the target class (Yes / No) in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV rows; initialize S with the first positive example and G with the all-'?' hypothesis.
4. For a positive example, replace the attributes of S that differ with '?' and remove from G the hypotheses that do not cover the example.
5. For a negative example, specialize every hypothesis in G that covers it using the values in S, and drop any hypothesis that is more general than another member of G.
6. Print S and G after each example; the last line gives the final version space.

Dataset File / Input

File name: fitness_data.csv

The CSV file contains a header row followed by these training examples:

Goal	Intensity	Location	Duration	ChooseActivity
Cardio	Moderate	Outdoor	30min	Yes
Cardio	High	Outdoor	30min	Yes
Flexibility	Low	Indoor	20min	No
Strength	Moderate	Indoor	45min	No
Cardio	Moderate	Outdoor	45min	Yes

Code

```
import csv

data = list(csv.reader(open("fitness_data.csv")))[1:]
n = len(data[0]) - 1
S = data[0][:-1]                      # first example is positive
G = [["?"] * n]

for row in data:
    x = row[:-1]
    if row[-1] == "Yes":
        S = [S[i] if S[i] == x[i] else "?" for i in range(n)]
        G = [g for g in G if all(g[i] in ("?", x[i]) for i in range(n))]
    else:
        new = []
        for g in G:
            if all(g[i] in ("?", x[i]) for i in range(n)):
                for i in range(n):
                    if g[i] == "?" and S[i] not in ("?", x[i]):
                        h = g.copy(); h[i] = S[i]; new.append(h)
            else:
                new.append(g)
        G = [g for g in new if not any(t != g and all(t[i] in ('?', g[i]) for i in range(n)) for t in new)]
    print("S =", S, " G =", G)
```

Output

```
... S = ['Cardio', 'Moderate', 'Outdoor', '30min']  G = [['?', '?', '?', '?']]
    S = ['Cardio', '?', 'Outdoor', '30min']  G = [['?', '?', '?', '?']]
    S = ['Cardio', '?', 'Outdoor', '30min']  G = [['Cardio', '?', '?', '?'], ['?', '?', 'Outdoor', '?'], ['?', '?', '?', '30min']]
    S = ['Cardio', '?', 'Outdoor', '30min']  G = [['Cardio', '?', '?', '?'], ['?', '?', 'Outdoor', '?'], ['?', '?', '?', '30min']]
    S = ['Cardio', '?', 'Outdoor', '?']  G = [['Cardio', '?', '?', '?'], ['?', '?', 'Outdoor', '?']]
```

Result

The final boundaries are S = ['Cardio', '?', 'Outdoor', '?'] and G = {['Cardio', '?', '?', '?'], ['?', '?', 'Outdoor', '?']}. Three hypotheses are consistent with the five training examples.